# Execution Context & the Call Stack

11 min read · Updated 17 Jul 2026 · Topic 7 of 23



Every line of JavaScript you write runs *inside* an execution context — an internal bookkeeping object the engine builds to track which variables exist, what `this` points to, and where the outer scope is. Interviewers probe this because it is the single mental model that explains hoisting, the TDZ, scope, `this`, closures, and "why did my `setTimeout` log last" all at once. If you can trace how contexts are created and stacked, most "what's the output?" trick questions become mechanical.

## What is an execution context?

An **execution context (EC)** is the environment in which a chunk of code is evaluated. There are three kinds:

- **Global execution context (GEC)** — created once when the script starts. It sets up the global object (`window` in browsers, `globalThis`/`global` in Node) and binds `this` to it (in non-module, non-strict code).
- **Function execution context (FEC)** — created *fresh every time a function is called*. Calling the same function three times makes three separate contexts with their own variables.
- **Eval execution context** — created by code run inside `eval()`. Rare, discouraged, and essentially never the intended answer in an interview — mention it exists and move on.

Each context internally holds three things: a **Variable Environment** (where `var` and function declarations live), a **Lexical Environment** (where `let`/`const` and the reference to the outer scope live), and the `this` **binding**.

## The two phases: creation, then execution

**Phase 1 — Creation (a.k.a. compilation/setup).** Before a single line runs, the engine scans the code and:

- Allocates memory for every `var` and initializes it to `undefined`.
- Allocates every function *declaration* fully (the whole function body is available immediately).
- Registers `let` / `const` names but leaves them **uninitialized** — this uninitialized window is the **Temporal Dead Zone (TDZ)**.
- Determines the scope chain (the link to the outer Lexical Environment).
- Binds `this`.

**Phase 2 — Execution.** Now the engine runs the code top to bottom, performing assignments and calling functions. This is why a `var` is `undefined` (declaration hoisted in phase 1, assignment happens in phase 2) but a function declaration is fully callable before its line:

JS Copy

```
console.log(x);    // undefined  (declaration hoisted, assignment not yet
console.log(fn()); // "hi"       (function declaration fully hoisted)
var x = 10;
function fn() { return "hi"; }
console.log(x);    // 10
```

`let` / `const` are *also* hoisted, but into the TDZ — touching them before their declaration throws instead of giving `undefined`:

JS Copy

```
try {
  console.log(a);  // ReferenceError - `a` is hoisted but in the TDZ
  let a = 1;
} catch (e) {
```

### Variable Environment vs Lexical Environment

Both are records of "what identifiers exist here," but the split matters: the **Variable Environment** holds `var`-declared and function-declared bindings, while the **Lexical Environment** holds `let`/`const` bindings and — crucially — the pointer to the *outer* environment. Historically `var` was function-scoped and lived in the Variable Environment for the whole function; `let`/`const` are block-scoped and get a fresh Lexical Environment per block. When people say "scope chain," they mean the chain of Lexical Environments linked by that outer pointer.

## The call stack

The **call stack** is the data structure the engine uses to keep track of *where it is*. It is a LIFO (last-in, first-out) stack of execution contexts:

- When the script starts, the **GEC is pushed** onto the stack.
- Every function **call pushes** a new FEC on top.
- When a function **returns** (or throws), its FEC is **popped** off.
- The context on top is the one currently executing. When the stack is empty, the program is done.

Because it is single-threaded, there is exactly **one** call stack, and only the top frame runs at any instant.

### Worked example: tracing nested calls

JS Copy

```
function first() {
  console.log("first: start");
  second();
  console.log("first: end");
}
function second() {
  console.log("second: start");
```

```
function third() {
  console.log("third");
}
first();
```

Output:

JAVASCRIPT Copy

```
first: start
second: start
third
second: end
first: end
```

Trace the stack. `first()` pushes `first`'s FEC. It logs `first: start`, then calls `second()` — which pushes `second`'s FEC *on top*, so `first` is now paused mid-execution. `second` logs, calls `third()` (pushed on top), `third` logs and **returns → popped**. Control resumes in `second` exactly where it paused; it logs `second: end` and **returns → popped**. Control resumes in `first`, which logs `first: end` and **returns → popped**. The outer `console.log`s (`first: end` after the inner ones) prove the outer frames were sitting on the stack, waiting, the whole time.

## Stack overflow: infinite recursion

Every un-returned call adds a frame. The stack has a finite size, so if calls never pop, you exhaust it and the engine throws `RangeError: Maximum call stack size exceeded`. Here it is demonstrated **safely** — a bounded recursion that works, and a runaway one caught with `try/catch` so it never crashes the process:

JS Copy

```
// Bounded — the same mechanism, but with a base case, so frames DO pop.
function depth(n) {
  if (n === 0) return 0;
  return 1 + depth(n - 1); // each call waits ⇒ a new frame stays on the
```

```
// No base case ⇒ frames never pop ⇒ the stack fills up.
function runaway() {
  return runaway();
}
try {
  runaway();
} catch (e) {
  console.log(e.name + ": " + e.message);
  // "RangeError: Maximum call stack size exceeded"
}
```

Note that `depth(n)` grows the stack to depth `n` too — the *only* difference from `runaway` is the base case that lets frames pop. The exact limit is engine- and frame-size-dependent (tens of thousands of frames), never a fixed number to quote.

## Scope chain vs call stack — the classic confusion

These two are constantly conflated, and interviewers love the distinction. **They answer different questions:**

- The **call stack** is about *runtime* — the dynamic order in which functions were *called*. It changes on every call/return.
- The **scope chain** is about *lexical structure* — where a function was *written* (defined) in the source. It is fixed when the function is created and never changes, regardless of who calls it.

A variable is resolved via the **scope chain (lexical), not the call stack.** This example nails it:

JS Copy

```
const x = "global";
function a() {
  const x = "a";
  b();            // b is CALLED from a (call stack: GEC → a → b) ...
}
```

```
function b() {
  // but b resolves x via its DEFINITION site ⟹ "glob
}
a();              // logs "global", NOT "a"
```

Even though `b` runs *on top of* `a` on the call stack, `b` was **defined** at the top level, so its outer environment is the global scope — it never sees `a`'s local `x`. JavaScript is lexically (statically) scoped; a language with dynamic scope would print `"a"`. This is also the foundation of closures: a function keeps a reference to the Lexical Environment where it was *defined*.

## `this` in the creation phase (brief)

During the creation phase, each context binds `this` — but the value is decided by **how the function is called**, not where it is defined (the mirror image of the scope-chain rule). A quick contrast:

JS Copy

```
"use strict";
function whoAmI() {
  console.log(this);
}
whoAmI();      // undefined  (plain call, strict mode)
const obj = { whoAmI };
obj.whoAmI(); // { whoAmI: [Function: whoAmI] }  (method call ⟹ the objec
```

In non-strict mode a plain call binds `this` to the global object instead of `undefined`. Arrow functions are the exception — they don't get their own `this` in creation; they close over the `this` of the surrounding context. That is a full topic on its own; here, just know `this` is one of the three things set up when a context is created.

## Single-threaded, synchronous execution

JavaScript has **one call stack and one thread** — it does exactly one thing at a time, and code runs to completion synchronously. The engine cannot pop a frame to "do

queued to run **later, only once the call stack is empty**, via the event loop.

JS Copy

```
console.log("A");
setTimeout(() => console.log("B (timeout)"), 0);
console.log("C");
// A
// C
// B (timeout)   ← runs after the stack is empty, even with a 0ms delay
```

`"B"` logs last despite the `0` ms delay because the synchronous `console.log("C")` must finish and the stack must drain before any queued callback is allowed to push a frame. That handoff — stack, task queue, microtask queue — is the **event loop**, the natural next topic.

## Common interview questions

**Q: What is an execution context and what are its phases?**
An internal environment the engine creates to run code; it holds the Variable Environment, Lexical Environment, and `this`. It is processed in two phases: **creation** (hoist `var`/functions, register `let`/`const` in the TDZ, set up scope and `this`) and **execution** (run statements, do assignments and calls).

**Q: How does the call stack work?**
It's a LIFO stack of execution contexts. The GEC is pushed at startup; each call pushes a new frame; each return pops one. Only the top frame executes. Empty stack = program finished.

**Q: What causes "Maximum call stack size exceeded"?**
Recursion (or mutual recursion) with no reachable base case — frames keep getting pushed and never pop, so the finite stack fills up and the engine throws a `RangeError`.

**Q: What's the difference between the scope chain and the call stack?**
The call stack is dynamic (call order, changes at runtime); the scope chain is lexical

**Q: If JavaScript is single-threaded, how does it do async work?**
The engine runs synchronously on one stack. Async callbacks (timers, I/O, promises) are handed to the host/runtime and queued; the event loop pushes a queued callback only when the stack is empty. Concurrency comes from the runtime, not from multiple JS threads.

**Q: Why is a `var` `undefined` before its line but a `let` throws?**
Both are hoisted in the creation phase, but `var` is initialized to `undefined`, whereas `let`/`const` are left uninitialized in the TDZ until execution reaches the declaration — reading them first throws a `ReferenceError`.

## Gotchas / traps

- `var` **hoists to** `undefined`**, not a** `ReferenceError`**.** `console.log(x); var x = 5;` prints `undefined`. Swap `var` for `let` and the same code throws `ReferenceError` (TDZ). Interviewers put both on the board.
- **A function *declaration* is fully hoisted; a function *expression* is not.** `foo();` works above `function foo(){}` but throws `TypeError: foo is not a function` above `var foo = function(){}` (because `foo` is hoisted as `undefined`, and you can't call `undefined`).
- **Scope is lexical, not dynamic.** The `a()`/`b()` example logs `"global"`. If you expected `"a"` because `b` was *called from* `a`, you conflated the call stack with the scope chain.
- `this` **follows the call site, not the definition.** Pulling a method off its object (`const f = obj.method; f();`) loses the object binding — `this` becomes `undefined` (strict) or global. The FEC binds `this` based on *how* `f` was invoked.
- `setTimeout(fn, 0)` **does not run** `fn` **"now."** It runs after the current synchronous code finishes and the stack drains. Any long synchronous loop before it delays it further — the `0` is a *minimum*, not a guarantee.
- **Each call is its own context.** Recursion isn't "one function reusing variables" — every invocation gets a fresh FEC with its own locals on its own frame. That's exactly why deep recursion consumes the stack.

- **The stack-size limit is not a number to memorize.** It depends on the engine and how much each frame holds (parameters, locals). Quoting "10,000" as a hard fact is a red flag; say "engine-dependent."

## QUICK CHECK — no marks, just for you.

1. In what order are these lines logged?

JAVASCRIPT Copy

```javascript
function outer() {
  console.log("outer:in");
  inner();
  console.log("outer:out");
}
function inner() {
  console.log("inner");
}
console.log("start");
outer();
console.log("end");
```

- ○ start<br>outer:in<br>outer:out<br>inner<br>end
- ○ start<br>outer:in<br>inner<br>outer:out<br>end
- ○ start<br>inner<br>outer:in<br>outer:out<br>end
- outer:in<br>inner

Check answer

2. What does this program log?

JAVASCRIPT Copy

```
const x = "global";
function a() {
  const x = "a";
  b();
}
function b() {
  console.log(x);
}
a();
```

- a
- undefined
- ReferenceError
- global

Check answer

3. When a JavaScript execution context is entered, what happens during the creation (first) phase, before any statement runs?

- Memory is allocated for every `var` (initialized to `undefined`) and for full function declarations, `let`/`const` names are registered but left uninitialized in the TDZ, and the scope chain and `this` are set up.

The engine runs the code top to bottom, performing assignments and invoking functions one statement at a time.

- Every `var` and `let` is assigned its source value and any function declarations are immediately invoked.
- Only the call stack is built; all variable bindings and `this` are deferred until the execution phase.

Check answer

‹ Previous
Scope, Hoisting & the TDZ

Next ›
Closures in Depth

## LEARN MORE

### ARTICLES & DOCS

- Call stack (MDN glossary)
  MDN
- The Modern JavaScript Tutorial
  javascript.info

### VIDEOS




- How JS is executed & the call stack
  YouTube




- How JS works under the hood
  YouTube




- Execution context & call stack
  YouTube

Hand-picked references for this lesson.

JavaScript Interview Kit